



awk

Последнее обновление: 21.03.2026

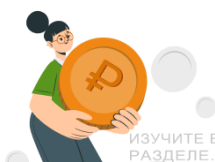


РЕКЛАМА

LIME

Быстрое решение

до 100 000 ₽ до 364 дней

**Взять займ**

ООО МФК «Лайм-Займ»

ИЗУЧИТЕ ВСЕ УСЛОВИЯ КРЕДИТА (ЗАЙМА) НА САЙТЕ В СООТВЕТСТВУЮЩЕМ РАЗДЕЛЕ. ОЦЕНИВАЙТЕ СВОИ ФИНАНСОВЫЕ ВОЗМОЖНОСТИ И РИСКИ.

Утилита **awk** представляет еще один мощный инструмент для работы с текстовыми файлами, предоставляя множество возможностей для обработки файлов. **awk**

Команда **awk** имеет следующий формат:

```
awk '/pattern/{action}' file
```

Сначала команде **awk** передается параметр **pattern** - шаблон (регулярное выражение), которому должен соответствовать текст. Обычно шаблоны заключаются между косыми чертами. Далее идет параметр **action**, который задает действия над найденным соответствием. Действия заключаются в фигурные скобки. Поскольку **awk** работает с регулярными выражениями, то шаблон разумно помещать в одинарные кавычки, чтобы оболочка случайно не интерпретировала их.

И в конце команде передается файл для поиска соответствий.

Если шаблон не указан, то действие выполняется над каждой строкой файла. Если действие не указано, **awk** просто показывает строки, соответствующие шаблону.

Например, пусть в домашнем каталоге пользователя у нас есть следующий файл **users.txt**

```
1 Tom:39:Microsoft
2 Bob:43:Google
3 Sam:28:lethrains
```

рейти



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**

Перейти



Условно говоря, каждая строка содержит данные пользователя - имя, возраст, компанию, то есть три **поля**. И каждое поле отделено от других двоеточием. То есть каждая строка имеет формат **имя:возраст:компания**

Допустим, нам надо получить все строки, где имя пользователя - "Tom":

```
eugene@Eugene:~$ awk '/Tom/' users.txt
Tom:39:Microsoft
Tomas:31:Apple
```

Утилита **awk** предоставляет ряд интересных возможностей, в частности, **awk** позволяет работать с файлами в табличной структурой, как наш файл. Например, для **awk** разделителем полей в строке по умолчанию является пробел. Однако в нашем файле таким разделителем является двоеточие. С помощью опции **-F** можно установить символ, который будет применяться в качестве разделителя.

Например, выведем только компанию, в которой работают пользователи с именем "Tom":

```
eugene@Eugene:~$ awk -F : '/Tom/{print $3}' users.txt
Microsoft
Apple
```

Здесь опции **-F** передается разделитель - двоеточие. Кроме того, здесь добавлено действие - встроенная команда **awk** - **print**, которая выводит передаваемое ей значение. Ей передается значение **\$3** - это значение 3-го столбца. В нашем случае 1-й столбец представляет имя, 2-й столбец - возраст и 3-й - компанию. Специальное значение - **\$0** означает всю строку.

Если бы мы хотели вывести компанию для всех пользователей, то можно было опустить шаблон:

```
eugene@Eugene:~$ awk -F : '{print $3}' users.txt
Microsoft
Google
Jetbrains
Microsoft
Google
-
Apple
```

Аналогично мы могли сослаться и на другие поля. Например, выведем сразу имя и компанию:

```
eugene@Eugene:~$ awk -F : '{print $1, $3}' users.txt
Tom Microsoft
Bob Google
Sam JetBrains
Alice Microsoft
```



Для обращения к последнему полю можно использовать специальный аргумент - `$NF` (NF – Number of Fields):

```
eugene@Eugene:~$ awk -F : '{print $1, $NF}' users.txt
Tom Microsoft
Bob Google
Sam JetBrains
Alice Microsoft
Kate Google
Tim -
Tomas Apple
```

Рассмотрим основные элементы встроенного языка `awk`.

Операторы

Поскольку `awk` может ссылаться на определенные поля, можно также использовать эти поля в различных операциях. `awk` поддерживает следующие операторы:

- **Арифметические:** `+`, `-`, `*`, `/`, `%` (модуль)
- **Операторы присваивания:** `=`, `+=`, `-=`, `*=`, `/=`, `%=`
- **Операторы сравнения:** `==`, `!=`, `>`, `<`, `>=`, `<=`
- **Логические операторы:** `&&` (И), `||` (ИЛИ), `!` (НЕ)
- **Условный оператор:** `?` : (тернарный оператор)

С помощью этих операторов вы можете выполнять некоторые вычисления с текстовыми файлами. Например, выведем пользователей, у которых возраст больше 30:

```
eugene@Eugene:~$ awk -F : '$2 > 30 {print $1}' users.txt
Tom
Bob
Alice
Tomas
```

Специальные переменные

`awk` поддерживает ряд специальных переменных:

- `$0`: представляет всю запись
- `NF` (Number of Fields): количество записей в файле
- `FS` (Field Separator): разделитель полей (по умолчанию: пробел/табуляция). Можно менять

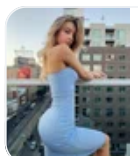
рейти



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**

Перейти



- NR (Number of Records): номер текущей строки (записи)
- RS (Record Separator): разделитель записей (по умолчанию: символ новой строки \n)
- ORS (Output Record Separator): разделитель записей в выводе (по умолчанию: \n)
- FILENAME: имя входного файла

Также **awk** позволяет использовать ряд переменных окружения:

- FNR: представляет номер записи в пределах текущего входного файла
- CONVFMT: используется для преобразования чисел в строку (% .6g по умолчанию)
- SUBSEP: разделитель, используемый для многомерных массивных индексов (по умолчанию "\034")
- OFMT: контролирует формат, используемый для числового вывода (%.6g по умолчанию)
- ARGV: хранит количество аргументов командной строки, переданных awk в виде массива
- ENVIRON: предоставляет доступ к переменным окружения
- RSTART: хранит начальную позицию наиболее недавнего совпадения или операции подстановки в awk
- RLENGTH: содержит длину наиболее недавнего совпадения или операции подстановки в awk
- ARGC: представляет индекс текущего входного файла, обрабатываемого awk
- IGNORECASE: указывает, надо ли игнорировать регистр при сопоставлении регулярных выражений
- ERRNO: хранит код ошибки, связанный с последним системным сбоем



Функции

awk поддерживает ряд функций, которые позволяют обрабатывать различные ситуации. Примеры функций:

- `index(s,t)`: находит позицию подстроки `t` в строке `s`

рейтин



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**

Перейти



- `match(s,r)`: ищет позицию совпадения регулярного выражения `r` в строке `s`
- `sub(r,t,s)`, `gsub(r,t,s)`: заменяют первое/все совпадения `r` на `t` в строке `s`
- `system(cmd)`: выполняет команду `cmd` и возвращает код выхода
- `srand([seed])`: Устанавливает `seed` для `rand` и возвращает предыдущий `seed`
- `split(s,a,fs)`: Разбивает строку `s` на массив `a`, используя `fs` в качестве разделителя
- `sprintf(fmt,...)`: Формирует строку `fmt` с указанными значениями
- `getline [var]`: Читает следующую запись из текущего входного файла
- `tolower(s)`: преобразуют строку в нижний регистр
- `toupper(s)`: преобразует строку `s` в верхний регистр

Управляющие конструкции

awk может рассматриваться как своего рода язык программирования, который поддерживает многие базовые конструкции управления потоком программы, а также работу с ассоциативными массивами и использование многомерных массивов. Рассмотрим некоторые примеры циклов и условных операторов.

Цикл For-in

```
1 awk 'BEGIN {
2     assoc["key1"] = "val1";
3     assoc["key2"] = "val2";
4     for (key in assoc)
5         print assoc[key];
6 }
```

Простой цикл For

```
1 awk 'BEGIN {
2     for (i = 0; i <= 10; i++)
3         print "i=", i;
4 }
```

Цикл While

```
1 awk 'BEGIN {
2     a = 1;
3     while (a < 10) {
4         print "-- concatenation:", a;
5         a++;
6     }
7 }
```

Цикл Do-While

рейти



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**

Перейти



```
6     i++;
7     } while (i <= 5)
8 }' /etc/passwd
```

Условный оператор If-Else

```
1 awk -v count=2 'BEGIN {
2     if (count == 1)
3         print "Yes";
4     else
5         print "Hm";
6 }
```

Альтернативная форма оператора If-Else:

```
1 awk -v count=2 'BEGIN {
2     print ((count == 1)? "Yes": "Hm");
3 }
```

Switch-case конструкция

```
1 awk '{
2     caseValue = $1;
3     switch (caseValue) {
4         case "apple":
5             print "Это яблоко!";
6             break;
7         case "banana":
8             print "Это банан";
9             break;
10        default:
11            print "Неизвестный фрукт";
12    }
13 }' fruits.txt
```

Одномерные массивы

```
1 awk 'BEGIN {
2     arr[0] = "foo";
3     arr[1] = "bar";
4     print(arr[0]); # => foo
5     delete arr[0];
6     print(arr[0]); # => ""
7 }
```

Многомерные массивы

```
1 awk 'BEGIN {
2     multidim[0,0] = "foo";
3     multidim[0,1] = "bar";
4     multidim[1,0] = "baz";
5     multidim[1,1] = "boo";
6 }
```

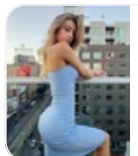
рейти



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**

Перейти



awk поддерживает синтаксис регулярных выражений, а также специальные символы, которые позволяют определить самые изощренные правила обработки файлов.

Применяемые управляющие последовательности

- \: Экранирующий символ
- \b: Возврат каретки
- \f: Перевод страницы
- \n: Новая строка
- \r: Возврат каретки
- \t: Горизонтальная табуляция
- \v: Вертикальная табуляция

Поддерживаемые элементы синтаксиса регулярных выражений:

- .: любой символ, кроме новой строки
- ?: 0 или 1 символ
- *: 0 или более символов
- +: 1 или более символов
- ^: Начало строки
- \$: Конец строки
- <: Пустая строка
- >: Конец слова

Спецификаторы форматирования:

- %c: ASCII-символ
- %d: Десятичное целое число
- %E, %e, %f: Число с плавающей точкой
- %o: Беззнаковое восьмеричное значение
- %s: Строка
- %%: символ %

Примеры использования команды **awk**

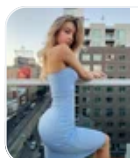
рейти



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**

Перейти



```

1 awk '{print $0}' file.txt # Вывести каждую строку файла
2 awk '/pattern/ {print $0}' file.txt # Вывести строки, соответствующие шаблону
3 awk '{print $1}' file.txt # Вывести первый столбец каждой строки
4 awk '{print $1, $3}' file.txt # Вывести первый и третий столбцы каждой строки
5 awk 'BEGIN {print "Header"} {print $0} END {print "Footer"}' file.txt # Добавить заголовок и

```

Разделители полей и записей

```

1 awk -F":" '{print $1}' /etc/passwd # Использовать «:» в качестве разделителя
2 awk 'BEGIN {FS=":"} {print $1}' /etc/passwd # Установить разделитель полей «:» в блоке
3 awk 'BEGIN {RS="\n\n"} {print $0}' file.txt # Использовать пустую строку в качестве ра

```

Поиск по шаблону

```

1 awk '$3 ~ /pattern/ {print $0}' file.txt # Вывести строки, где третье поле соответс
2 awk '$3 !~ /pattern/ {print $0}' file.txt # Вывести строки, где третье поле не соотв

```

Арифметические операции

```

1 awk '{print $1 + $2}' file.txt # Сложить первое и второе поля каждой строки
2 awk '{sum += $1} END {print sum}' file.txt # Посчитать сумму значений первого поля во
3 awk '{avg = ($1 + $2 + $3)/3; print avg}' file.txt # Посчитать среднее значение первых трёх п

```

Условные операторы

```

1 awk '{if ($1 > 50) print $0}' file.txt # Вывести строки, где первое поле больше 50
2 awk '$1 > 50 {print $0}' file.txt # Альтернативный синтаксис для того же условия
3 awk '{if ($1 > 50) print "High"; else print "Low"}' file.txt # Вывести "High" или "Low" в за

```

Работа с несколькими файлами

```

1 awk 'FNR==1 {print "Processing " FILENAME}' file1.txt file2.txt # Вывести сообщение при начал
2 awk 'FNR==NR {sum1 += $1; next} {sum2 += $1} END {print sum1, sum2}' file1.txt file2.txt # Ра

```

рейтин



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**

Перейти



[Настройки](#)[Помощь сайту](#)[Помощь сайту](#)

Юмани:

410011174743222

Номер карты:

4048415020898850



care.yandex.ru РЕКЛАМА

**Отменили оплату ОСАГО
за год — платите
помесячно**

Оформите ОСАГО один раз и забудьте
о продлении. Кешбэк 100% за первый
месяц

[Оформить ОСАГО онлайн](#) >[Удобно и безопасно](#) >[Заказать](#)

2026. Все права защищены.

рейтин



rusmeet.com РЕКЛАМА

**Бесплатные знакомства
с настоящими женщинами**[Перейти](#)